UK Road Traffic Accidents API Documentation

Submission-ready API documentation export derived from the repository specification and verified against generated OpenAPI metadata.

SOURCE	GENERATED	PATHS	OPERATIONS
api-spec.md	**21 March 2026**	**34**	**43**

Contents

Project Overview
Base URL
Conventions
CRUD Endpoints
Weather Enrichment Endpoints
Relationship Endpoints
Cluster Endpoints
Analytical Endpoints
Route Risk

UK Road Traffic Accidents — API Specification

Project Overview

A data-driven REST API built on **FastAPI** and **PostgreSQL** that exposes the STATS19 UK road accident dataset (2019–2023, ~500,000 records) enriched with **Met Office MIDAS** weather observations, pre-computed **DBSCAN spatial clusters**, and a **route risk scoring** engine.

Total documented endpoints: **43**.

The API is organised into six capability layers:

Layer	Endpoints	Purpose
CRUD	16	Full read/write access to accident, vehicle, and casualty records
Weather Enrichment	2	Weather station resource exposing MIDAS observation data linked to accidents
Relationship	4	Region → local authority hierarchy and scoped accident collections
Cluster	3	Pre-computed DBSCAN spatial clusters as a first-class resource
Analytical	16	Aggregation and cross-tabulation endpoints
Route Risk	2	On-demand route scoring and scoring-model transparency

The three extended pillars — weather enrichment, cluster resource, and route risk scoring — collectively enable the API to reason about road risk rather than just retrieve accident records.

Base URL

```
/api/v1
```

Conventions

Response Envelope

Collection

```
{
  "data": [ ... ],
  "meta": { "page": 1, "per_page": 25, "total": 4821 }
}
```

Single resource

```
{ "data": { ... } }
```

Analytical

```
{ "data": [ ... ], "query": { "<applied parameters>" } }
```

Scoped collection (used when a child resource collection is scoped to a parent)

```
{
  "context": { "<parent resource>" },
  "data": [ ... ],
  "meta": { "page": 1, "per_page": 25, "total": 312 }
}
```

Error

```
{ "error": { "code": "NOT_FOUND", "message": "Accident not found.", "details": [] } }
```

Path Parameter Conventions

`:id`

The primary identifier for top-level resources (accidents, regions, local authorities). For accidents this is the STATS19 accident index string (e.g. `2022010012345`). For other resources it is an integer primary key.

`:ref`

An ordinal reference number scoped within a parent accident, not a global database ID. This mirrors the STATS19 source data structure in which vehicles and casualties are numbered sequentially per accident (1, 2, 3...). A `:ref` value is only meaningful in combination with its parent `:id`.

Content-Type

All requests with a body must include the header:

```
Content-Type: application/json
```

All responses carry `Content-Type: application/json`.

Authentication and Authorization

- `GET` endpoints are public for coursework demonstration.
- Write endpoints (`POST`, `PATCH`, `DELETE` under `/accidents` and child resources) require `Authorization: Bearer <JWT>`.
- Role model:
- `editor`: create/update records
- `admin`: delete records
- Requests with missing/invalid token return `401`.
- Requests with insufficient role return `403`.

HTTP Status Codes

Code	Meaning
`200`	Successful GET, PUT, PATCH
`201`	Successful POST — resource created
`204`	Successful DELETE — no response body
`401`	Missing or invalid authentication token
`403`	Authenticated but not authorized for the operation
`400`	Malformed request or invalid parameter value
`404`	Resource not found
`422`	Well-formed request that is semantically invalid
`500`	Unhandled server error

Common Query Parameters (Collections)

Parameter	Type	Default	Description
`page`	integer	`1`	Page number
`per_page`	integer	`25`	Results per page, max `100`
`sort`	string	`date`	Sort field
`order`	string	`desc`	`asc` or `desc`

CRUD Endpoints

`GET /accidents`

Purpose: Return a paginated, filterable list of accident records.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Inclusive lower date bound
`date_to`	`YYYY-MM-DD`	Inclusive upper date bound
`severity`	integer	`1` Fatal, `2` Serious, `3` Slight
`region_id`	integer	Filter to a region — adds an implicit JOIN via `local_authority`
`local_authority_id`	integer	Filter to a local authority
`road_type_id`	integer	Road type foreign key
`weather_condition_id`	integer	Weather condition foreign key
`light_condition_id`	integer	Light condition foreign key
`speed_limit`	integer	Exact speed limit in mph
`urban_or_rural`	string	`Urban` or `Rural`
`cluster_id`	integer	Filter to accidents belonging to a specific DBSCAN cluster
`sort`	string	`date` (default) or `severity`
`order`	string	`asc` or `desc` (default)

Response Schema:

Field	Type	Nullable	Description
`id`	string	No	STATS19 accident index
`date`	string	No	ISO 8601 date
`time`	string	Yes	HH:MM:SS
`day_of_week`	integer	No	1 (Sunday) to 7 (Saturday)
`latitude`	float	No	WGS84 latitude
`longitude`	float	No	WGS84 longitude
`severity`	object	No	`{ id, label }`
`speed_limit`	integer	Yes	Speed limit in mph
`urban_or_rural`	string	Yes	`Urban`, `Rural`, or `Unallocated`
`number_of_vehicles`	integer	No	Vehicle count — maintained by the application, incremented on vehicle add
`number_of_casualties`	integer	No	Casualty count — maintained by the application, incremented on casualty add
`local_authority`	object	Yes	`{ id, name }`
`region`	object	Yes	`{ id, name }`

Example Response `200 OK`:

```
{
  "data": [
    {
      "id": "2022010012345",
      "date": "2022-03-15",
      "time": "08:42:00",
      "day_of_week": 2,
      "latitude": 53.8008,
      "longitude": -1.5491,
      "severity": { "id": 2, "label": "Serious" },
      "speed_limit": 30,
      "urban_or_rural": "Urban",
      "number_of_vehicles": 2,
      "number_of_casualties": 1,
      "local_authority": { "id": 14, "name": "Leeds" },
      "region": { "id": 3, "name": "Yorkshire and The Humber" }
    }
  ],
  "meta": { "page": 1, "per_page": 25, "total": 4821 }
}
```

Status Codes: `200` `400` `422`

`GET /accidents/:id`

Purpose: Return full detail for a single accident including all dimension labels and embedded vehicle and casualty records.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index

Response Schema:

All fields from the list response, plus:

Field	Type	Nullable	Description
`road_type`	object	Yes	`{ id, label }`
`junction_detail`	object	Yes	`{ id, label }`
`light_condition`	object	Yes	`{ id, label }`
`weather_condition`	object	Yes	`{ id, label }`
`road_surface`	object	Yes	`{ id, label }`
`police_attended`	boolean	Yes	Whether police attended the scene
`cluster_id`	integer	Yes	DBSCAN cluster this accident belongs to, or `null` if noise point
`weather_observation`	object	Yes	MIDAS observed weather at the time and nearest station — see below
`vehicles`	array	No	Embedded vehicle records
`casualties`	array	No	Embedded casualty records

`weather_observation` object:

Field	Type	Nullable	Description
`station_id`	integer	No	MIDAS weather station ID
`station_name`	string	No	Station name
`station_distance_km`	float	No	Distance from accident to station in km
`temperature_c`	float	Yes	Air temperature in °C
`precipitation_mm`	float	Yes	Precipitation in mm over the observation hour
`wind_speed_ms`	float	Yes	Wind speed in m/s
`visibility_m`	integer	Yes	Visibility in metres

Example Response `200 OK`:

```
{
  "data": {
    "id": "2022010012345",
    "date": "2022-03-15",
    "time": "08:42:00",
    "day_of_week": 2,
    "latitude": 53.8008,
    "longitude": -1.5491,
    "severity": { "id": 2, "label": "Serious" },
    "road_type": { "id": 3, "label": "Single carriageway" },
    "junction_detail": { "id": 3, "label": "T or staggered junction" },
    "light_condition": { "id": 1, "label": "Daylight" },
    "weather_condition": { "id": 2, "label": "Raining without strong winds" },
    "road_surface": { "id": 2, "label": "Wet or damp" },
    "speed_limit": 30,
    "urban_or_rural": "Urban",
    "number_of_vehicles": 2,
    "number_of_casualties": 1,
    "police_attended": true,
    "cluster_id": 14,
    "weather_observation": {
      "station_id": 3802,
      "station_name": "Leeds Bradford Airport",
      "station_distance_km": 4.2,
      "temperature_c": 8.1,
      "precipitation_mm": 1.4,
      "wind_speed_ms": 5.2,
      "visibility_m": 4800
    },
    "local_authority": { "id": 14, "name": "Leeds" },
    "region": { "id": 3, "name": "Yorkshire and The Humber" },
    "vehicles": [
      {
        "vehicle_ref": 1,
        "vehicle_type": { "id": 9, "label": "Car" },
        "age_of_driver": 34,
        "sex_of_driver": "Male",
        "propulsion_code": "Petrol",
        "age_of_vehicle": 5,
        "journey_purpose": "Commute"
      }
    ],
    "casualties": [
      {
        "casualty_ref": 1,
        "vehicle_ref": 1,
        "severity": { "id": 2, "label": "Serious" },
        "casualty_class": "Driver",
        "casualty_type": "Car occupant",
        "sex": "Male",
        "age": 34,
```

```
        "age_band": "26-35"
      }
    ]
  }
}
```

Status Codes: `200` `404`

> **Implementation note:** The `vehicles` and `casualties` arrays must be populated using two separate queries (`SELECT ... WHERE accident_id = ?` for each child table), not a single JOIN across all three tables. A triple JOIN produces a cartesian product (M vehicles × N casualties rows) that requires complex application-side deduplication. Issue the accident query first, then fetch vehicles and casualties in parallel if supported by the ORM, and merge in application code.

`POST /accidents`

Purpose: Create a new accident record. Vehicles and casualties are added separately via their own endpoints.

Request Body:

Field	Type	Required	Description
`date`	string	Yes	`YYYY-MM-DD`
`time`	string	No	`HH:MM:SS`
`day_of_week`	integer	Yes	1–7
`latitude`	float	Yes	WGS84 latitude
`longitude`	float	Yes	WGS84 longitude
`severity_id`	integer	Yes	FK to severity
`road_type_id`	integer	No	FK to road_type
`junction_detail_id`	integer	No	FK to junction_detail
`light_condition_id`	integer	No	FK to light_condition
`weather_condition_id`	integer	No	FK to weather_condition
`road_surface_id`	integer	No	FK to road_surface
`speed_limit`	integer	No	Speed limit in mph
`local_authority_id`	integer	No	FK to local_authority
`urban_or_rural`	string	No	`Urban`, `Rural`, or `Unallocated`
`police_attended`	boolean	No	Whether police attended

> `number_of_vehicles` and `number_of_casualties` are not accepted as inputs. They are stored as integer columns on the accident row, initialised to `0` on creation, and maintained by the application: incremented when a vehicle or casualty is added via `POST`, decremented when one is deleted.

Example Request Body:

```
{
  "date": "2024-11-01",
  "time": "17:15:00",
  "day_of_week": 5,
  "latitude": 51.5074,
  "longitude": -0.1278,
  "severity_id": 3,
  "road_type_id": 3,
  "speed_limit": 30,
  "local_authority_id": 1,
  "urban_or_rural": "Urban",
  "police_attended": false
}
```

Example Response `201 Created`:

```
{
  "data": {
    "id": "2024010098765",
    "date": "2024-11-01",
    "time": "17:15:00",
    "day_of_week": 5,
    "latitude": 51.5074,
    "longitude": -0.1278,
    "severity": { "id": 3, "label": "Slight" },
    "speed_limit": 30,
    "urban_or_rural": "Urban",
    "number_of_vehicles": 0,
    "number_of_casualties": 0,
    "police_attended": false,
    "local_authority": { "id": 1, "name": "Westminster" },
    "region": { "id": 7, "name": "London" }
  }
}
```

Status Codes: `201` `400` `401` `403` `422`

`PATCH /accidents/:id`

Purpose: Partially update an accident record. Only fields provided are modified. Used for targeted corrections such as updating severity after investigation.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index

Request Body: Any subset of fields accepted by `POST /accidents`.

> `number_of_vehicles` and `number_of_casualties` are not patchable. Submitting either field returns `422 Unprocessable Entity`.

Example Request Body:

```
{ "severity_id": 1 }
```

Example Response `200 OK`:

```
{
  "data": {
    "id": "2022010012345",
    "severity": { "id": 1, "label": "Fatal" }
  }
}
```

Status Codes: `200` `400` `401` `403` `404` `422`

`DELETE /accidents/:id`

Purpose: Permanently delete an accident and cascade-delete all associated vehicle and casualty records.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index

Response: `204 No Content` (empty body)

Status Codes: `204` `401` `403` `404`

`GET /accidents/:id/vehicles`

Purpose: List all vehicles involved in a specific accident.

Response Schema:

Field	Type	Nullable	Description
`vehicle_ref`	integer	No	Ordinal reference within the accident (1-based)
`vehicle_type`	object	Yes	`{ id, label }`
`age_of_driver`	integer	Yes	Driver age in years
`sex_of_driver`	string	Yes	`Male`, `Female`, or `Not known`
`engine_capacity_cc`	integer	Yes	Engine capacity
`propulsion_code`	string	Yes	e.g. `Petrol`, `Diesel`, `Electric`
`age_of_vehicle`	integer	Yes	Vehicle age in years
`journey_purpose`	string	Yes	e.g. `Commute`, `Recreation`

Example Response `200 OK`:

```
{
  "data": [
    {
      "vehicle_ref": 1,
      "vehicle_type": { "id": 9, "label": "Car" },
      "age_of_driver": 34,
      "sex_of_driver": "Male",
      "engine_capacity_cc": 1600,
      "propulsion_code": "Petrol",
      "age_of_vehicle": 5,
      "journey_purpose": "Commute"
    },
    {
      "vehicle_ref": 2,
      "vehicle_type": { "id": 2, "label": "Motorcycle over 500cc" },
      "age_of_driver": 27,
      "sex_of_driver": "Male",
      "engine_capacity_cc": 650,
      "propulsion_code": "Petrol",
      "age_of_vehicle": 2,
      "journey_purpose": "Recreation"
    }
  ]
}
```

Status Codes: `200` `404`

`GET /accidents/:id/vehicles/:ref`

Purpose: Return a single vehicle by its ordinal reference within an accident.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index
`ref`	integer	Vehicle reference number within the accident

Example Response `200 OK`:

```
{
  "data": {
    "vehicle_ref": 1,
    "vehicle_type": { "id": 9, "label": "Car" },
    "age_of_driver": 34,
    "sex_of_driver": "Male",
    "engine_capacity_cc": 1600,
    "propulsion_code": "Petrol",
    "age_of_vehicle": 5,
    "journey_purpose": "Commute"
  }
}
```

Status Codes: `200` `404`

`POST /accidents/:id/vehicles`

Purpose: Add a vehicle record to an existing accident. `vehicle_ref` is assigned automatically as the next ordinal.

Request Body:

Field	Type	Required	Description
`vehicle_type_id`	integer	No	FK to vehicle_type
`age_of_driver`	integer	No	Driver age in years
`sex_of_driver`	string	No	`Male`, `Female`, or `Not known`
`engine_capacity_cc`	integer	No	Engine capacity
`propulsion_code`	string	No	Fuel type
`age_of_vehicle`	integer	No	Vehicle age in years
`journey_purpose`	string	No	Purpose of journey

Example Request Body:

```
{
  "vehicle_type_id": 9,
  "age_of_driver": 45,
  "sex_of_driver": "Female",
  "propulsion_code": "Diesel",
  "age_of_vehicle": 3,
  "journey_purpose": "Other"
}
```

Example Response `201 Created`:

```
{
  "data": {
    "vehicle_ref": 3,
    "vehicle_type": { "id": 9, "label": "Car" },
    "age_of_driver": 45,
    "sex_of_driver": "Female",
    "propulsion_code": "Diesel",
    "age_of_vehicle": 3,
    "journey_purpose": "Other"
  }
}
```

Status Codes: `201` `400` `401` `403` `404`

`PATCH /accidents/:id/vehicles/:ref`

Purpose: Partially update a vehicle record. Only fields provided are modified.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index
`ref`	integer	Vehicle reference number

Request Body: Any subset of fields accepted by `POST /accidents/:id/vehicles`.

Example Response `200 OK`: Updated vehicle object.

Status Codes: `200` `400` `401` `403` `404`

`DELETE /accidents/:id/vehicles/:ref`

Purpose: Remove a vehicle record from an accident.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index
`ref`	integer	Vehicle reference number

Response: `204 No Content`

> If casualties currently reference the deleted `vehicle_ref`, the API nulls those casualty `vehicle_ref` values in the same transaction before deleting the vehicle.

Status Codes: `204` `401` `403` `404`

`GET /accidents/:id/casualties`

Purpose: List all casualties recorded in a specific accident.

Response Schema:

Field	Type	Nullable	Description
`casualty_ref`	integer	No	Ordinal reference within the accident
`vehicle_ref`	integer	Yes	Links casualty to a vehicle in the same accident
`severity`	object	No	`{ id, label }`
`casualty_class`	string	Yes	`Driver`, `Passenger`, or `Pedestrian`
`casualty_type`	string	Yes	e.g. `Car occupant`, `Cyclist`, `Pedestrian`
`sex`	string	Yes	`Male`, `Female`, or `Not known`
`age`	integer	Yes	Age in years
`age_band`	string	Yes	e.g. `26-35`

Example Response `200 OK`:

```
{
  "data": [
    {
      "casualty_ref": 1,
      "vehicle_ref": 1,
      "severity": { "id": 2, "label": "Serious" },
      "casualty_class": "Driver",
      "casualty_type": "Car occupant",
      "sex": "Male",
      "age": 34,
      "age_band": "26-35"
    }
  ]
}
```

Status Codes: `200` `404`

`GET /accidents/:id/casualties/:ref`

Purpose: Return a single casualty by its ordinal reference within an accident.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index
`ref`	integer	Casualty reference number

Example Response `200 OK`:

```
{
  "data": {
    "casualty_ref": 1,
    "vehicle_ref": 1,
    "severity": { "id": 2, "label": "Serious" },
    "casualty_class": "Driver",
    "casualty_type": "Car occupant",
    "sex": "Male",
    "age": 34,
    "age_band": "26-35"
  }
}
```

Status Codes: `200` `404`

`POST /accidents/:id/casualties`

Purpose: Add a casualty record to an existing accident.

Request Body:

Field	Type	Required	Description
`vehicle_ref`	integer	No	Links to a vehicle in this accident
`severity_id`	integer	Yes	FK to severity
`casualty_class`	string	No	`Driver`, `Passenger`, or `Pedestrian`
`casualty_type`	string	No	e.g. `Car occupant`, `Cyclist`
`sex`	string	No	`Male`, `Female`, or `Not known`
`age`	integer	No	Age in years

> `age_band` is not accepted as an input. It is derived server-side from `age` using the STATS19 standard band definitions.

Example Request Body:

```
{
  "vehicle_ref": 1,
  "severity_id": 3,
  "casualty_class": "Passenger",
  "casualty_type": "Car occupant",
  "sex": "Female",
  "age": 29
}
```

Example Response `201 Created`:

```
{
  "data": {
    "casualty_ref": 2,
    "vehicle_ref": 1,
    "severity": { "id": 3, "label": "Slight" },
    "casualty_class": "Passenger",
    "casualty_type": "Car occupant",
    "sex": "Female",
    "age": 29,
    "age_band": "26-35"
  }
}
```

Status Codes: `201` `400` `401` `403` `404` `422`

`PATCH /accidents/:id/casualties/:ref`

Purpose: Partially update a casualty record. Only fields provided are modified.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index
`ref`	integer	Casualty reference number

Request Body: Any subset of fields accepted by `POST /accidents/:id/casualties`.

Example Response `200 OK`: Updated casualty object.

Status Codes: `200` `400` `401` `403` `404`

`DELETE /accidents/:id/casualties/:ref`

Purpose: Remove a casualty record from an accident.

Path Parameters:

Parameter	Type	Description
`id`	string	STATS19 accident index
`ref`	integer	Casualty reference number

Response: `204 No Content`

Status Codes: `204` `401` `403` `404`

`GET /reference/conditions`

Purpose: Return all lookup tables in a single response. Used to populate filter dropdowns without multiple round-trips. Scope to one table using the `type` parameter.

> **Design note:** This endpoint intentionally departs from strict REST resource modelling by returning multiple resource types in one response. The trade-off is justified: clients initialising a filter UI require all condition tables simultaneously, and separate per-table endpoints would produce unnecessary round-trips with no benefit. The `type` parameter preserves the ability to fetch a single table when needed.

Query Parameters:

Parameter	Type	Description
`type`	string	Optional. One of: `weather`, `light`, `road_surface`, `road_type`, `junction`, `vehicle_type`

Example Response `200 OK`:

```
{
  "data": {
    "severities": [
      { "id": 1, "label": "Fatal" },
      { "id": 2, "label": "Serious" },
      { "id": 3, "label": "Slight" }
    ],
    "weather_conditions": [
      { "id": 1, "label": "Fine without strong winds" },
      { "id": 2, "label": "Raining without strong winds" },
      { "id": 3, "label": "Snowing without strong winds" },
      { "id": 6, "label": "Fog or mist" }
    ],
    "light_conditions": [
      { "id": 1, "label": "Daylight" },
      { "id": 4, "label": "Darkness: street lighting present and lit" },
      { "id": 5, "label": "Darkness: street lighting present but unlit" },
      { "id": 6, "label": "Darkness: no street lighting" }
    ],
    "road_surfaces": [
      { "id": 1, "label": "Dry" },
      { "id": 2, "label": "Wet or damp" },
      { "id": 3, "label": "Snow" },
      { "id": 4, "label": "Frost or ice" }
    ],
    "road_types": [
      { "id": 1, "label": "Roundabout" },
      { "id": 3, "label": "Dual carriageway" },
      { "id": 6, "label": "Single carriageway" }
    ],
    "junction_details": [
      { "id": 0, "label": "Not at junction or within 20 metres" },
      { "id": 1, "label": "Roundabout" },
      { "id": 3, "label": "T or staggered junction" },
      { "id": 6, "label": "Crossroads" }
    ],
    "vehicle_types": [
      { "id": 1, "label": "Pedal cycle" },
      { "id": 9, "label": "Car" },
      { "id": 11, "label": "Bus or coach" },
      { "id": 20, "label": "Truck / HGV" }
    ]
  }
}
```


Status Codes: `200` `400`

Weather Enrichment Endpoints

During data import, each accident record is matched to the nearest Met Office MIDAS weather station (within 30km) and the closest hourly observation to the accident time. This links real measured atmospheric conditions — temperature, precipitation, wind, visibility — to accident records, extending analysis beyond STATS19's pre-coded categorical weather field.

`GET /weather-stations`

Purpose: List all MIDAS weather stations present in the enriched dataset. Only stations that contributed at least one observation linked to an accident are returned.

Query Parameters:

Parameter	Type	Description
`region_id`	integer	Scope to stations within a region's bounding box
`active_on`	`YYYY-MM-DD`	Return only stations active on this date

Response Schema:

Field	Type	Nullable	Description
`id`	integer	No	MIDAS station identifier
`name`	string	No	Station name
`latitude`	float	No	WGS84 latitude
`longitude`	float	No	WGS84 longitude
`elevation_m`	integer	Yes	Station elevation in metres above sea level
`active_from`	string	Yes	ISO 8601 date the station became active
`active_to`	string	Yes	ISO 8601 date the station was decommissioned, or `null` if still active
`linked_accident_count`	integer	No	Number of accidents linked to this station

Example Response `200 OK`:

```
{
  "data": [
    {
      "id": 3802,
      "name": "Leeds Bradford Airport",
      "latitude": 53.8693,
      "longitude": -1.6604,
      "elevation_m": 208,
      "active_from": "1959-01-01",
      "active_to": null,
      "linked_accident_count": 4218
    },
    {
      "id": 3011,
      "name": "Ringway",
      "latitude": 53.3536,
      "longitude": -2.2772,
      "elevation_m": 75,
      "active_from": "1949-01-01",
      "active_to": null,
      "linked_accident_count": 8941
    }
  ],
  "meta": { "page": 1, "per_page": 25, "total": 187 }
}
```

Status Codes: `200` `400`

`GET /weather-stations/:id`

Purpose: Return a single weather station by ID, including summary statistics of the observations it contributed to the dataset.

Path Parameters:

Parameter	Type	Description
`id`	integer	MIDAS station ID

Example Response `200 OK`:

```
{
  "data": {
    "id": 3802,
    "name": "Leeds Bradford Airport",
    "latitude": 53.8693,
    "longitude": -1.6604,
    "elevation_m": 208,
    "active_from": "1959-01-01",
    "active_to": null,
    "linked_accident_count": 4218,
    "observation_summary": {
      "mean_temperature_c": 9.4,
      "mean_precipitation_mm": 0.14,
      "mean_wind_speed_ms": 4.8,
      "mean_visibility_m": 18200,
      "observations_with_precipitation": 1204
    }
  }
}
```

Status Codes: `200` `404`

Relationship Endpoints

`GET /regions`

Purpose: List all regions.

Example Response `200 OK`:

```
{
  "data": [
    { "id": 1, "name": "North East England", "local_authority_count": 12 },
    { "id": 2, "name": "North West England", "local_authority_count": 39 },
    { "id": 3, "name": "Yorkshire and The Humber", "local_authority_count": 21 },
    { "id": 7, "name": "London", "local_authority_count": 33 }
  ]
}
```

Status Codes: `200`

`GET /regions/:id`

Purpose: Return a single region by ID.

Path Parameters:

Parameter	Type	Description
`id`	integer	Region ID

Example Response `200 OK`:

```
{
  "data": {
    "id": 3,
    "name": "Yorkshire and The Humber",
    "local_authority_count": 21
  }
}
```

Status Codes: `200` `404`

`GET /regions/:id/local-authorities`

Purpose: List all local authorities that belong to a region, using the scoped collection envelope with the region as context.

Path Parameters:

Parameter	Type	Description
`id`	integer	Region ID

Example Response `200 OK`:

```
{
  "context": { "id": 3, "name": "Yorkshire and The Humber" },
  "data": [
    { "id": 14, "name": "Leeds" },
    { "id": 15, "name": "Bradford" },
    { "id": 16, "name": "Sheffield" },
    { "id": 17, "name": "Wakefield" }
  ]
}
```

Status Codes: `200` `404`

`GET /local-authorities/:id/accidents`

Purpose: Return accidents scoped to a specific local authority, with the local authority context prepended to the response. Accepts the same filter and pagination parameters as `GET /accidents`.

Path Parameters:

Parameter	Type	Description
`id`	integer	Local authority ID

Query Parameters: Same as `GET /accidents`, excluding `local_authority_id`.

Example Response `200 OK`:

```
{
  "context": {
    "id": 14,
    "name": "Leeds",
    "region": { "id": 3, "name": "Yorkshire and The Humber" }
  },
  "data": [
    {
      "id": "2022010012345",
      "date": "2022-03-15",
      "severity": { "id": 2, "label": "Serious" },
      "speed_limit": 30,
      "number_of_vehicles": 2,
      "number_of_casualties": 1
    }
  ],
  "meta": { "page": 1, "per_page": 25, "total": 312 }
}
```

Status Codes: `200` `404`

Cluster Endpoints

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is run as a **post-import preprocessing step** over all accident coordinates using `epsilon = 0.008°` (~890m) and `min_samples = 10`. The resulting cluster assignments are stored in a `cluster` table and in a `cluster_id` column on each accident row, making clusters a queryable first-class resource rather than something computed at request time.

Accidents not assigned to any cluster (DBSCAN noise points) have `cluster_id = null`.

`GET /clusters`

Purpose: List all pre-computed DBSCAN spatial clusters, sorted by accident count descending. Each cluster represents a statistically significant geographic concentration of accidents.

Query Parameters:

Parameter	Type	Description
`region_id`	integer	Scope to clusters whose centroid falls within a region
`min_accidents`	integer	Minimum accident count, default `10`
`severity_label`	string	Filter by cluster severity label: `Low`, `Medium`, `High`, or `Critical`

Response Schema:

Field	Type	Nullable	Description
`id`	integer	No	Cluster ID
`centroid_lat`	float	No	Centroid latitude
`centroid_lng`	float	No	Centroid longitude
`radius_km`	float	No	Approximate radius enclosing 95% of cluster members
`accident_count`	integer	No	Total accidents in this cluster
`fatal_count`	integer	No	Fatal accidents
`serious_count`	integer	No	Serious accidents
`fatal_rate_pct`	float	No	Fatal accidents as percentage of total
`severity_label`	string	No	`Low` (<1% fatal), `Medium` (1–2%), `High` (2–5%), `Critical` (>5%)
`local_authority`	object	Yes	Local authority whose boundary contains the centroid

Example Response `200 OK`:

```
{
  "data": [
    {
      "id": 14,
      "centroid_lat": 53.8012,
      "centroid_lng": -1.5489,
      "radius_km": 0.82,
      "accident_count": 214,
      "fatal_count": 6,
      "serious_count": 38,
      "fatal_rate_pct": 2.80,
      "severity_label": "High",
      "local_authority": { "id": 14, "name": "Leeds" }
    },
    {
      "id": 91,
      "centroid_lat": 51.5021,
      "centroid_lng": -0.1289,
      "radius_km": 1.14,
      "accident_count": 389,
      "fatal_count": 3,
      "serious_count": 51,
      "fatal_rate_pct": 0.77,
      "severity_label": "Low",
      "local_authority": { "id": 1, "name": "Westminster" }
    }
  ],
  "meta": { "page": 1, "per_page": 25, "total": 1842 }
}
```

Status Codes: `200` `400`

`GET /clusters/:id`

Purpose: Return full detail for a single cluster including bounding box, dominant conditions, and year-on-year accident count trend.

Path Parameters:

Parameter	Type	Description
`id`	integer	Cluster ID

Response Schema:

All fields from the list response, plus:

Field	Type	Nullable	Description
`bbox`	object	No	`{ min_lat, min_lng, max_lat, max_lng }` — bounding box of cluster members
`dominant_conditions`	object	No	Most common weather, light, road surface, and speed limit among cluster accidents
`annual_trend`	array	No	Year-by-year accident count within the cluster

Example Response `200 OK`:

```
{
  "data": {
    "id": 14,
    "centroid_lat": 53.8012,
    "centroid_lng": -1.5489,
    "radius_km": 0.82,
    "accident_count": 214,
    "fatal_count": 6,
    "serious_count": 38,
    "fatal_rate_pct": 2.80,
    "severity_label": "High",
    "local_authority": { "id": 14, "name": "Leeds" },
    "bbox": { "min_lat": 53.794, "min_lng": -1.561, "max_lat": 53.809, "max_lng": -1.537 },
    "dominant_conditions": {
      "weather": "Fine without strong winds",
      "light": "Darkness: street lighting present and lit",
      "road_surface": "Dry",
      "speed_limit": 30
    },
    "annual_trend": [
      { "year": 2019, "accident_count": 48 },
      { "year": 2020, "accident_count": 29 },
      { "year": 2021, "accident_count": 41 },
      { "year": 2022, "accident_count": 52 },
      { "year": 2023, "accident_count": 44 }
    ]
  }
}
```

Status Codes: `200` `404`

`GET /clusters/:id/accidents`

Purpose: Return all accidents belonging to a specific cluster, using the scoped collection envelope. Accepts the same filter and pagination parameters as `GET /accidents`.

Path Parameters:

Parameter	Type	Description
`id`	integer	Cluster ID

Query Parameters: Same as `GET /accidents`, excluding `cluster_id`.

Example Response `200 OK`:

```
{
  "context": {
    "id": 14,
    "centroid_lat": 53.8012,
    "centroid_lng": -1.5489,
    "severity_label": "High"
  },
  "data": [
    {
      "id": "2022010012345",
      "date": "2022-03-15",
      "severity": { "id": 2, "label": "Serious" },
      "speed_limit": 30,
      "number_of_vehicles": 2,
      "number_of_casualties": 1
    }
  ],
  "meta": { "page": 1, "per_page": 25, "total": 214 }
}
```

Status Codes: `200` `404`

Analytical Endpoints

All analytical endpoints return a `query` object echoing the parameters that were applied.

`GET /analytics/accidents-by-time`

Purpose: Return accident frequency bucketed by hour of day (0–23) and day of week (1–7). The full 168-cell matrix is returned including cells with zero accidents so clients can render a complete heatmap without inferring gaps.

> **Implementation note:** SQL `GROUP BY` only produces rows for cells that have data. The zero-fill must be applied in Python after the query: build a 24×7 dict pre-populated with `accident_count: 0`, merge the query result in, then flatten to a list ordered by day then hour.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`severity`	integer	Filter to a severity level
`region_id`	integer	Scope to a region

Response Schema:

Field	Type	Description
`hour`	integer	Hour of day, 0–23
`day_of_week`	integer	1 (Sunday) to 7 (Saturday)
`day_label`	string	Human-readable day name
`accident_count`	integer	Number of accidents in this cell

Example Response `200 OK`:

```
{
  "data": [
    { "hour": 0,  "day_of_week": 1, "day_label": "Sunday",   "accident_count": 18 },
    { "hour": 8,  "day_of_week": 2, "day_label": "Tuesday",  "accident_count": 142 },
    { "hour": 17, "day_of_week": 5, "day_label": "Friday",   "accident_count": 189 },
    { "hour": 23, "day_of_week": 7, "day_label": "Saturday", "accident_count": 34 }
  ],
  "query": { "date_from": "2019-01-01", "date_to": "2023-12-31", "severity": null,
"region_id": null }
}
```

Status Codes: `200` `400` `422`

`GET /analytics/annual-trend`

Purpose: Return year-on-year totals for accidents, total casualties, and fatal casualties with percentage change from the prior year.

Query Parameters:

Parameter	Type	Description
`year_from`	integer	Start year (inclusive)
`year_to`	integer	End year (inclusive)
`region_id`	integer	Scope to a region
`local_authority_id`	integer	Scope to a local authority

Response Schema:

Field	Type	Description
`year`	integer	Calendar year
`accidents`	integer	Total accidents
`casualties`	integer	Total casualties
`fatal_casualties`	integer	Casualties with fatal severity
`change_pct`	float or null	Percentage change vs prior year; `null` for first year

Example Response `200 OK`:

```
{
  "data": [
    { "year": 2019, "accidents": 153158, "casualties": 184469, "fatal_casualties": 1752,
"change_pct": null },
    { "year": 2020, "accidents": 91199,  "casualties": 109138, "fatal_casualties": 1472,
"change_pct": -40.4 },
    { "year": 2021, "accidents": 101087, "casualties": 121609, "fatal_casualties": 1558,
"change_pct": 10.8 }
  ],
  "query": { "year_from": 2019, "year_to": 2023, "region_id": null }
}
```

Status Codes: `200` `400` `422`

`GET /analytics/severity-by-conditions`

Purpose: Cross-tabulate accident severity against one environmental condition dimension. The `dimension` parameter selects the grouping axis, replacing the need for separate endpoints per condition type. MIDAS-enriched dimensions (`precipitation_band` , `visibility_band` , `temperature_band`) use binned continuous values from observed weather data rather than STATS19's categorical codes.

> **Implementation note:** The dimensions `weather` , `light` , `road_surface` , `road_type` , and `junction` are stored as foreign keys and require a JOIN to the corresponding lookup table. `urban_or_rural` is a plain string column grouped directly without a JOIN. MIDAS dimensions (`precipitation_band` , `visibility_band` ,

`temperature_band`) join to the `weather_observation` table and apply a `CASE` expression to bin continuous values. The query builder must branch on which dimension is requested.

Query Parameters:

Parameter	Type	Required	Description
`dimension`	string	Yes	`weather` , `light` , `road_surface` , `road_type` , `junction` , `urban_or_rural` , `precipitation_band` , `visibility_band` , or `temperature_band`
`date_from`	`YYYY-MM-DD`	No	Start of date range
`date_to`	`YYYY-MM-DD`	No	End of date range
`region_id`	integer	No	Scope to a region

> **MIDAS dimension bands:** `precipitation_band` bins as `Dry (<0.2mm)` , `Light (0.2–2mm)` , `Moderate (2–10mm)` , `Heavy (>10mm)` . `visibility_band` bins as `Dense Fog (<100m)` , `Fog (100–1000m)` , `Mist (1000–5000m)` , `Clear (>5000m)` . `temperature_band` bins as `Freezing (<=0°C)` , `Cold (0–7°C)` , `Mild (7–15°C)` , `Warm (>15°C)` . Only accidents with a linked `weather_observation` are included when using MIDAS dimensions.

Response Schema:

Field	Type	Description
`condition`	string	Label of the condition value
`fatal`	integer	Fatal accidents
`serious`	integer	Serious accidents
`slight`	integer	Slight accidents
`total`	integer	Total accidents
`fatal_rate_pct`	float	Percentage of total that are fatal
`query.coverage_pct`	float	Percent of accidents in scope with MIDAS coverage (non-MIDAS dimensions return `100.0`)

Example Response `200 OK` :

```
{
  "data": [
    { "condition": "Fine without strong winds", "fatal": 892, "serious": 12041, "slight":
94312, "total": 107245, "fatal_rate_pct": 0.83 },
    { "condition": "Fog or mist",               "fatal": 41,  "serious": 490,   "slight":
2890,  "total": 3421,   "fatal_rate_pct": 1.20 },
    { "condition": "Snowing without strong winds", "fatal": 12, "serious": 319, "slight":
2104,  "total": 2435,   "fatal_rate_pct": 0.49 }
  ],
  "query": { "dimension": "weather", "date_from": null, "date_to": null, "region_id": null,
"coverage_pct": 100.0 }
}
```


Status Codes: `200` `400` `422`

`GET /analytics/severity-by-speed-limit`

Purpose: Return severity distribution and fatal rate for each speed limit band, enabling comparison of injury outcomes across road speed environments.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`urban_or_rural`	string	`Urban` or `Rural`
`region_id`	integer	Scope to a region

Response Schema:

Field	Type	Description
`speed_limit`	integer	Speed limit in mph
`total_accidents`	integer	Total accidents at this speed limit
`fatal`	integer	Fatal accidents
`serious`	integer	Serious accidents
`slight`	integer	Slight accidents
`fatal_rate_pct`	float	Fatal accidents as percentage of total
`avg_casualties_per_accident`	float	Mean casualties per accident

Example Response `200 OK`:

```
{
  "data": [
    { "speed_limit": 20, "total_accidents": 8912,  "fatal": 14,  "serious": 901,  "slight":
7997,  "fatal_rate_pct": 0.16, "avg_casualties_per_accident": 1.08 },
    { "speed_limit": 30, "total_accidents": 74121, "fatal": 412, "serious": 9201, "slight":
64508, "fatal_rate_pct": 0.56, "avg_casualties_per_accident": 1.19 },
    { "speed_limit": 70, "total_accidents": 19041, "fatal": 612, "serious": 4201, "slight":
14228, "fatal_rate_pct": 3.21, "avg_casualties_per_accident": 1.31 }
  ],
  "query": { "urban_or_rural": null, "region_id": null }
}
```

Status Codes: `200` `400`

`GET /analytics/hotspots`

Purpose: Return accident density for all grid cells within a radius of a given coordinate, grouped into 0.01-degree cells. This is an **ad-hoc spatial query** intended for exploring arbitrary locations. For pre-identified high-risk concentrations, use `GET /clusters` which exposes DBSCAN-computed clusters as a first-class resource with richer metadata and better performance.

> **Implementation note:** The bounding box pre-filter (`lat BETWEEN ? AND ?` , `lng BETWEEN ? AND ?`) runs in SQL using standard B-tree indexes. The exact radius check uses the Haversine formula, which requires `SIN` , `COS` , `ASIN` , and `SQRT` — functions available in PostgreSQL but not in SQLite's default build (SQLite requires compilation with `SQLITE_ENABLE_MATH_FUNCTIONS`). This endpoint requires PostgreSQL. The radius check can alternatively be applied in Python after the bounding-box SQL query.

Query Parameters:

Parameter	Type	Required	Description
`lat`	float	Yes	Centre latitude (WGS84)
`lng`	float	Yes	Centre longitude (WGS84)
`radius_km`	float	No	Search radius in km, default `5`
`severity`	integer	No	Filter to a severity level
`date_from`	`YYYY-MM-DD`	No	Start of date range
`date_to`	`YYYY-MM-DD`	No	End of date range

Response Schema:

Field	Type	Description
`cell_lat`	float	Grid cell centre latitude
`cell_lng`	float	Grid cell centre longitude
`accident_count`	integer	Accidents within this cell
`fatal_count`	integer	Fatal accidents within this cell
`serious_count`	integer	Serious accidents within this cell

Example Response `200 OK`:

```
{
  "data": [
    { "cell_lat": 53.80, "cell_lng": -1.55, "accident_count": 47, "fatal_count": 2,
"serious_count": 11 },
    { "cell_lat": 53.81, "cell_lng": -1.54, "accident_count": 31, "fatal_count": 0,
"serious_count": 7  }
  ],
  "query": { "lat": 53.8008, "lng": -1.5491, "radius_km": 5, "severity": null }
}
```

Status Codes: `200` `400` `422`

`GET /analytics/accidents-by-vehicle-type`

Purpose: Return accident counts segmented by vehicle type with severity breakdown, showing which vehicle categories appear most frequently in serious or fatal collisions.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`region_id`	integer	Scope to a region

Response Schema:

Field	Type	Description
`vehicle_type`	string	Vehicle type label
`accidents_involved_in`	integer	Accidents where this vehicle type appeared
`fatal_count`	integer	Of those, accidents with at least one fatal casualty
`serious_count`	integer	Of those, accidents with at least one serious casualty
`fatal_rate_pct`	float	Fatal rate as a percentage

Example Response `200 OK`:

```
{
  "data": [
    { "vehicle_type": "Car",                  "accidents_involved_in": 112041, "fatal_count":
1201, "serious_count": 18400, "fatal_rate_pct": 1.07 },
    { "vehicle_type": "Pedal cycle",          "accidents_involved_in": 18241,  "fatal_count":
94,   "serious_count": 3912,  "fatal_rate_pct": 0.52 },
    { "vehicle_type": "Motorcycle over 500cc","accidents_involved_in": 9801,   "fatal_count":
198,  "serious_count": 3100,  "fatal_rate_pct": 2.02 }
  ],
  "query": { "date_from": null, "date_to": null, "region_id": null }
}
```

Status Codes: `200` `400`

`GET /analytics/casualties-by-demographic`

Purpose: Break down casualties by age band, casualty class, and sex to identify which demographic groups are over-represented in accident casualties.

Query Parameters:

Parameter	Type	Description
`severity`	integer	Filter to a severity level
`casualty_type`	string	e.g. `Pedestrian`, `Cyclist`, `Car occupant`
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range

Response Schema:

Field	Type	Description
`age_band`	string	Age band e.g. `16-20`, `26-35`, `75+`
`casualty_class`	string	`Driver`, `Passenger`, or `Pedestrian`
`sex`	string	`Male`, `Female`, or `Not known`
`count`	integer	Number of casualties in this group
`pct_of_total`	float	Percentage of all casualties in the result set

Example Response `200 OK`:

```
{
  "data": [
    { "age_band": "16-20", "casualty_class": "Driver",     "sex": "Male",   "count": 4921,
"pct_of_total": 3.2 },
    { "age_band": "75+",   "casualty_class": "Pedestrian", "sex": "Female", "count": 1203,
"pct_of_total": 0.8 }
  ],
  "query": { "severity": 1, "casualty_type": null }
}
```

Status Codes: `200` `400`

`GET /analytics/fatal-condition-combinations`

Purpose: Identify which combinations of environmental conditions carry the highest fatal accident rate. Returns condition combinations (weather × light × road surface × junction type) ranked by `fatal_rate_pct` rather than raw count, preventing common conditions from dominating results simply due to exposure volume. A `min_count` threshold filters out statistically insignificant combinations.

Query Parameters:

Parameter	Type	Description
`year_from`	integer	Start year
`year_to`	integer	End year
`region_id`	integer	Scope to a region
`min_count`	integer	Minimum `total_accidents` for a combination to appear, default `10`
`limit`	integer	Number of results, default `20`

Response Schema:

Field	Type	Description
`rank`	integer	Rank by fatal rate percentage
`weather`	string	Weather condition label
`light`	string	Light condition label
`road_surface`	string	Road surface label
`junction_detail`	string	Junction type label
`total_accidents`	integer	Total accidents matching this combination
`fatal_accidents`	integer	Fatal accidents matching this combination
`fatal_rate_pct`	float	Fatal accidents as percentage of total for this combination

Example Response `200 OK`:

```
{
  "data": [
    {
      "rank": 1,
      "weather": "Fine without strong winds",
      "light": "Darkness: no street lighting",
      "road_surface": "Dry",
      "junction_detail": "Not at junction or within 20 metres",
      "total_accidents": 4201,
      "fatal_accidents": 189,
      "fatal_rate_pct": 4.50
    },
    {
      "rank": 2,
      "weather": "Fog or mist",
      "light": "Darkness: street lighting present and lit",
      "road_surface": "Wet or damp",
      "junction_detail": "Not at junction or within 20 metres",
      "total_accidents": 812,
      "fatal_accidents": 34,
      "fatal_rate_pct": 4.19
    }
  ],
  "query": { "year_from": 2019, "year_to": 2023, "region_id": null, "min_count": 10, "limit":
20 }
}
```

Status Codes: `200` `400`

`GET /analytics/accidents-by-local-authority`

Purpose: Return all local authorities ranked by total accident count over a period with severity breakdown, enabling comparison of accident prevalence across administrative areas.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`severity`	integer	Filter to a severity level
`region_id`	integer	Scope to a region
`limit`	integer	Number of results, default `20`

Response Schema:

Field	Type	Description
`local_authority`	object	`{ id, name, region }`
`total_accidents`	integer	Total accidents in the period
`fatal_accidents`	integer	Fatal accidents
`serious_accidents`	integer	Serious accidents
`fatal_rate_pct`	float	Fatal accidents as percentage of total

Example Response `200 OK`:

```
{
  "data": [
    { "local_authority": { "id": 1,  "name": "Birmingham", "region": "West Midlands" },
"total_accidents": 8941, "fatal_accidents": 42, "serious_accidents": 901, "fatal_rate_pct":
0.47 },
    { "local_authority": { "id": 14, "name": "Leeds",      "region": "Yorkshire and The
Humber" }, "total_accidents": 6812, "fatal_accidents": 31, "serious_accidents": 712,
"fatal_rate_pct": 0.45 }
  ],
  "meta": { "total_authorities": 326 },
  "query": { "date_from": "2019-01-01", "date_to": "2023-12-31", "region_id": null }
}
```

Status Codes: `200` `400` `422`

`GET /analytics/severity-by-journey-purpose`

Purpose: Break down accident severity by the stated journey purpose of vehicle drivers involved, revealing which travel contexts (commuting, leisure, professional driving) produce the highest rates of serious or fatal injury.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`vehicle_type_id`	integer	Scope to a specific vehicle type
`region_id`	integer	Scope to a region

Response Schema:

Field	Type	Description
`journey_purpose`	string	Journey purpose label
`total_accidents`	integer	Accidents involving a vehicle with this journey purpose
`fatal`	integer	Fatal accidents
`serious`	integer	Serious accidents
`slight`	integer	Slight accidents
`fatal_rate_pct`	float	Fatal accidents as percentage of total
`serious_or_fatal_rate_pct`	float	Serious or fatal accidents as percentage of total

Example Response `200 OK`:


```
{
  "data": [
    { "journey_purpose": "Commute",       "total_accidents": 41201, "fatal": 312, "serious":
5901, "slight": 34988, "fatal_rate_pct": 0.76, "serious_or_fatal_rate_pct": 15.07 },
    { "journey_purpose": "Recreation",    "total_accidents": 18412, "fatal": 241, "serious":
3100, "slight": 15071, "fatal_rate_pct": 1.31, "serious_or_fatal_rate_pct": 18.16 },
    { "journey_purpose": "Professional",  "total_accidents": 9801,  "fatal": 198, "serious":
1801, "slight": 7802,  "fatal_rate_pct": 2.02, "serious_or_fatal_rate_pct": 20.39 },
    { "journey_purpose": "Other",         "total_accidents": 12041, "fatal": 89,  "serious":
1201, "slight": 10751, "fatal_rate_pct": 0.74, "serious_or_fatal_rate_pct": 10.72 }
  ],
  "query": { "date_from": null, "date_to": null, "vehicle_type_id": null, "region_id": null }
}
```


Status Codes: `200` `400`

`GET /analytics/seasonal-pattern`

Purpose: Return accident count and fatal rate broken down by month of year (1–12), aggregated across all years in the dataset. Exposes seasonal variation driven by road conditions (winter ice/fog raising fatal rates) and traffic volume (summer leisure traffic raising counts).

Query Parameters:

Parameter	Type	Description
`year_from`	integer	Start year (inclusive)
`year_to`	integer	End year (inclusive)
`region_id`	integer	Scope to a region
`urban_or_rural`	string	`Urban` or `Rural`

Response Schema:

Field	Type	Description
`month`	integer	Month number, 1 (January) to 12 (December)
`month_label`	string	Month name
`total_accidents`	integer	Total accidents in this month across all years
`fatal_accidents`	integer	Fatal accidents
`fatal_rate_pct`	float	Fatal accidents as percentage of total
`avg_accidents_per_year`	float	Mean annual accidents for this month

Example Response `200 OK`:


```
{
  "data": [
    { "month": 1,  "month_label": "January",   "total_accidents": 62841, "fatal_accidents":
621, "fatal_rate_pct": 0.99, "avg_accidents_per_year": 12568 },
    { "month": 7,  "month_label": "July",       "total_accidents": 71203, "fatal_accidents":
589, "fatal_rate_pct": 0.83, "avg_accidents_per_year": 14241 },
    { "month": 12, "month_label": "December",   "total_accidents": 67412, "fatal_accidents":
698, "fatal_rate_pct": 1.04, "avg_accidents_per_year": 13482 }
  ],
  "query": { "year_from": 2019, "year_to": 2023, "region_id": null, "urban_or_rural": null }
}
```


Status Codes: `200` `400`

`GET /analytics/driver-age-severity`

Purpose: Cross-tabulate accident severity against driver age bands. The join traverses `accident → vehicle` rather than accident-only, demonstrating a multi-table analytical query. Reveals which age groups are involved in the most serious collisions.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`vehicle_type_id`	integer	Scope to a specific vehicle type
`region_id`	integer	Scope to a region

Response Schema:

Field	Type	Description
`age_band`	string	Driver age band, e.g. `17-24`, `25-34`, `65+`
`total_accidents`	integer	Accidents involving a driver in this age band
`fatal`	integer	Fatal accidents
`serious`	integer	Serious accidents
`slight`	integer	Slight accidents
`fatal_rate_pct`	float	Fatal accidents as percentage of total
`serious_or_fatal_rate_pct`	float	Serious or fatal accidents as percentage of total

Example Response `200 OK`:

```
{
  "data": [
    { "age_band": "17-24", "total_accidents": 29801, "fatal": 312, "serious": 5201, "slight":
24288, "fatal_rate_pct": 1.05, "serious_or_fatal_rate_pct": 18.50 },
    { "age_band": "25-34", "total_accidents": 41203, "fatal": 289, "serious": 5901, "slight":
35013, "fatal_rate_pct": 0.70, "serious_or_fatal_rate_pct": 15.03 },
    { "age_band": "65+",   "total_accidents": 14201, "fatal": 241, "serious": 2801, "slight":
11159, "fatal_rate_pct": 1.70, "serious_or_fatal_rate_pct": 21.42 }
  ],
  "query": { "date_from": null, "date_to": null, "vehicle_type_id": null, "region_id": null }
}
```

Status Codes: `200` `400`

`GET /analytics/vulnerable-road-users`

Purpose: Analyse casualty outcomes for pedestrians and cyclists only, broken down by speed limit and urban/rural classification. Traverses `casualty → accident` (the reverse direction to most endpoint queries), exposing systemic risk to non-vehicle road users.

Query Parameters:

Parameter	Type	Description
`casualty_type`	string	`Pedestrian` or `Cyclist` — if omitted, both are included
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`region_id`	integer	Scope to a region

Response Schema:

Field	Type	Description
`speed_limit`	integer	Speed limit in mph
`urban_or_rural`	string	`Urban`, `Rural`, or `Unallocated`
`total_casualties`	integer	Casualties of the specified type in this context
`fatal_casualties`	integer	Fatal casualties
`serious_casualties`	integer	Serious casualties
`fatal_rate_pct`	float	Fatal casualties as percentage of total

Example Response `200 OK`:

```
{
  "data": [
    { "speed_limit": 20, "urban_or_rural": "Urban",  "total_casualties": 4201,
"fatal_casualties": 4,   "serious_casualties": 512, "fatal_rate_pct": 0.10 },
    { "speed_limit": 30, "urban_or_rural": "Urban",  "total_casualties": 39012,
"fatal_casualties": 298, "serious_casualties": 6801, "fatal_rate_pct": 0.76 },
    { "speed_limit": 60, "urban_or_rural": "Rural",  "total_casualties": 8901,
"fatal_casualties": 201, "serious_casualties": 2100, "fatal_rate_pct": 2.26 }
  ],
  "query": { "casualty_type": "Pedestrian", "date_from": null, "date_to": null, "region_id":
null }
}
```

Status Codes: `200` `400` `422`

`GET /analytics/police-attendance-profile`

Purpose: Compare severity distributions of accidents where police attended versus those where they did not. `police_attended` in STATS19 correlates strongly with perceived severity at point of report; this endpoint tests that assumption and quantifies the gap.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`region_id`	integer	Scope to a region

Response Schema:

Field	Type	Description
`police_attended`	boolean	Whether police attended the scene
`total_accidents`	integer	Total accidents in this group
`fatal`	integer	Fatal accidents
`serious`	integer	Serious accidents
`slight`	integer	Slight accidents
`fatal_rate_pct`	float	Fatal accidents as percentage of total
`serious_or_fatal_rate_pct`	float	Serious or fatal accidents as percentage of total

Example Response `200 OK`:

```
{
  "data": [
    { "police_attended": true,  "total_accidents": 289041, "fatal": 3891, "serious": 51201,
"slight": 233949, "fatal_rate_pct": 1.35, "serious_or_fatal_rate_pct": 19.06 },
    { "police_attended": false, "total_accidents": 124812, "fatal": 201,  "serious": 8901,
"slight": 115710, "fatal_rate_pct": 0.16, "serious_or_fatal_rate_pct": 7.30 }
  ],
  "query": { "date_from": null, "date_to": null, "region_id": null }
}
```

Status Codes: `200` `400`

`GET /analytics/multi-vehicle-severity`

Purpose: Compare severity profiles and average casualty counts between single-vehicle accidents and multi-vehicle collisions, optionally broken down by speed limit. Single-vehicle fatals (run-off-road, pedestrian strikes with no other

party) have different causal profiles from multi-vehicle collisions and warrant separate analysis.

Query Parameters:

Parameter	Type	Description
`date_from`	`YYYY-MM-DD`	Start of date range
`date_to`	`YYYY-MM-DD`	End of date range
`group_by_speed_limit`	boolean	If `true`, adds a `speed_limit` breakdown dimension — default `false`
`region_id`	integer	Scope to a region

Response Schema:

Field	Type	Description
`collision_type`	string	`Single vehicle` or `Multi vehicle`
`speed_limit`	integer or null	Speed limit if `group_by_speed_limit=true`, otherwise `null`
`total_accidents`	integer	Total accidents
`fatal`	integer	Fatal accidents
`serious`	integer	Serious accidents
`slight`	integer	Slight accidents
`fatal_rate_pct`	float	Fatal accidents as percentage of total
`avg_casualties_per_accident`	float	Mean number of casualties per accident

Example Response `200 OK`:


```
{
  "data": [
    { "collision_type": "Single vehicle", "speed_limit": null, "total_accidents": 89412,
"fatal": 1891, "serious": 12401, "slight": 75120, "fatal_rate_pct": 2.11,
"avg_casualties_per_accident": 1.06 },
    { "collision_type": "Multi vehicle",  "speed_limit": null, "total_accidents": 324441,
"fatal": 2201, "serious": 46801, "slight": 275439, "fatal_rate_pct": 0.68,
"avg_casualties_per_accident": 1.32 }
  ],
  "query": { "date_from": null, "date_to": null, "group_by_speed_limit": false, "region_id":
null }
}
```


Status Codes: `200` `400`

`GET /analytics/weather-correlation`

Purpose: Cross-tabulate accident severity against binned MIDAS observed weather metrics (precipitation, visibility, temperature, wind speed). Unlike `severity-by-conditions` which uses STATS19's categorical weather codes, this endpoint uses measured values from linked weather station observations — enabling quantitative statements such as "accidents in precipitation >10mm are X times more likely to be fatal than in dry conditions". Only accidents with a linked weather observation are included.

Query Parameters:

Parameter	Type	Required	Description
`metric`	string	Yes	`precipitation`, `visibility`, `temperature`, or `wind_speed`
`date_from`	`YYYY-MM-DD`	No	Start of date range
`date_to`	`YYYY-MM-DD`	No	End of date range
`region_id`	integer	No	Scope to a region

Response Schema:

Field	Type	Description
`band`	string	Metric band label
`band_range`	string	Numeric range of the band, e.g. `2–10mm`
`total_accidents`	integer	Accidents in this band (with observation data)
`fatal`	integer	Fatal accidents
`serious`	integer	Serious accidents
`slight`	integer	Slight accidents
`fatal_rate_pct`	float	Fatal accidents as percentage of total
`coverage_pct`	float	Percentage of all matched accidents this band represents

Example Response `200 OK`:

```
{
  "data": [
    { "band": "Dry",      "band_range": "<0.2mm",  "total_accidents": 241082, "fatal": 1891,
"serious": 35201, "slight": 203990, "fatal_rate_pct": 0.78, "coverage_pct": 71.2 },
    { "band": "Light",    "band_range": "0.2–2mm", "total_accidents": 54301,  "fatal": 512,
"serious": 8901,  "slight": 44888,  "fatal_rate_pct": 0.94, "coverage_pct": 16.0 },
    { "band": "Moderate", "band_range": "2–10mm",  "total_accidents": 31201,  "fatal": 389,
"serious": 5801,  "slight": 25011,  "fatal_rate_pct": 1.25, "coverage_pct": 9.2 },
    { "band": "Heavy",    "band_range": ">10mm",   "total_accidents": 12041,  "fatal": 210,
"serious": 2401,  "slight": 9430,   "fatal_rate_pct": 1.74, "coverage_pct": 3.6 }
  ],
  "query": { "metric": "precipitation", "date_from": null, "date_to": null, "region_id": null
}
}
```


Status Codes: `200` `400` `422`

Route Risk

The route risk engine is the API's flagship analytical feature. Given a road route as an ordered sequence of waypoints, it segments the route at fixed intervals, scores each segment against the historical accident database using a multi-factor weighted model, and returns a per-segment risk profile alongside an aggregate route score.

This is not a retrieval endpoint. It executes a scoring computation backed by the database and returns a derived result.

`POST /analytics/route-risk`

Purpose: Score a road route for historical accident risk. The route is decomposed into segments of `segment_length_km`. For each segment midpoint, five risk factors are computed from the database and combined into a score in `[0, 1]`. The response includes per-segment detail and an aggregate route summary.

Request Body:

Field	Type	Required	Description
`waypoints`	array	Yes	Ordered array of `[latitude, longitude]` pairs defining the route. Minimum 2, maximum 500.
`options.time_of_day`	string	No	`HH:MM` — used to look up time-of-day risk from the accidents-by-time heatmap. Defaults to current UTC time.
`options.day_of_week`	integer	No	`1` – `7` (1 = Sunday). Defaults to current day.
`options.segment_length_km`	float	No	Segment interval in km, default `0.5`, min `0.1`, max `2.0`.
`options.buffer_radius_km`	float	No	Radius around each segment midpoint to search for accidents, default `0.5`.

Example Request Body:

```
{
  "waypoints": [
    [53.8008, -1.5491],
    [53.8124, -1.5312],
    [53.8201, -1.5098]
  ],
  "options": {
    "time_of_day": "08:30",
    "day_of_week": 2,
    "segment_length_km": 0.5
  }
}
```

Response Schema — `route_summary`:

Field	Type	Description
`total_distance_km`	float	Total route length in km
`segment_count`	integer	Number of scored segments
`aggregate_risk_score`	float	Mean risk score across all segments, `0` – `1`
`risk_label`	string	`Very Low`, `Low`, `Moderate`, `High`, or `Critical`
`peak_segment_risk`	float	Highest single-segment risk score
`peak_segment_id`	integer	1-based index of the highest-risk segment
`clusters_intersected`	integer	Number of DBSCAN clusters whose radius the route passes through

Response Schema — `segments[]`:

Field	Type	Description
`segment_id`	integer	1-based ordinal
`start`	array	`[lat, lng]` of segment start
`end`	array	`[lat, lng]` of segment end
`length_km`	float	Actual segment length
`risk_score`	float	Composite risk score `0` – `1`
`risk_label`	string	`Very Low` / `Low` / `Moderate` / `High` / `Critical`
`factors`	object	Individual normalised scores for each factor (see below)
`nearby_accidents`	integer	Accidents within the buffer radius
`nearby_cluster_ids`	array	IDs of DBSCAN clusters intersecting this segment's buffer
`dominant_speed_limit`	integer	Most common speed limit among nearby accidents

`factors` object:

Field	Type	Description
`accident_density`	float	`0` – `1` — accident count per km² normalised against dataset maximum
`severity_score`	float	`0` – `1` — severity-weighted mean of nearby accidents
`time_risk`	float	`0` – `1` — from accidents-by-time heatmap for requested hour and day
`speed_limit_risk`	float	`0` – `1` — fatal rate for the dominant speed limit, normalised
`cluster_proximity`	float	`0` – `1` — `1.0` if inside a cluster, decays with distance otherwise

Example Response `200 OK`:

```
{
  "data": {
    "route_summary": {
      "total_distance_km": 2.8,
      "segment_count": 6,
      "aggregate_risk_score": 0.44,
      "risk_label": "Moderate",
      "peak_segment_risk": 0.71,
      "peak_segment_id": 3,
      "clusters_intersected": 1
    },
    "segments": [
      {
        "segment_id": 1,
        "start": [53.8008, -1.5491],
        "end":   [53.8051, -1.5432],
        "length_km": 0.50,
        "risk_score": 0.31,
        "risk_label": "Low",
        "factors": {
          "accident_density": 0.28,
          "severity_score": 0.22,
          "time_risk": 0.61,
          "speed_limit_risk": 0.18,
          "cluster_proximity": 0.00
        },
        "nearby_accidents": 8,
        "nearby_cluster_ids": [],
        "dominant_speed_limit": 30
      },
      {
        "segment_id": 3,
        "start": [53.8101, -1.5358],
        "end":   [53.8144, -1.5269],
        "length_km": 0.50,
        "risk_score": 0.71,
        "risk_label": "High",
        "factors": {
          "accident_density": 0.82,
          "severity_score": 0.68,
          "time_risk": 0.61,
          "speed_limit_risk": 0.18,
          "cluster_proximity": 1.00
        },
        "nearby_accidents": 47,
        "nearby_cluster_ids": [14],
        "dominant_speed_limit": 30
      }
    ]
  },
  "query": {
```

```
    "waypoint_count": 3,
    "segment_length_km": 0.5,
    "buffer_radius_km": 0.5,
    "time_of_day": "08:30",
    "day_of_week": 2
  }
}
```

Status Codes: `200` `400` `422`

> **Note on `200`:** This endpoint returns a computed analytical result and does not persist a new server-side resource.

`GET /analytics/route-risk/scoring-model`

Purpose: Return the scoring formula, factor weights, and risk label thresholds used by `POST /analytics/route-risk`. This transparency endpoint allows clients to explain and validate scores.

Example Response `200 OK`:

```
{
  "data": {
    "formula": "risk_score = w1*accident_density + w2*severity_score + w3*time_risk +
w4*speed_limit_risk + w5*cluster_proximity",
    "weights": {
      "accident_density": 0.35,
      "severity_score": 0.30,
      "time_risk": 0.15,
      "speed_limit_risk": 0.10,
      "cluster_proximity": 0.10
    },
    "factor_descriptions": {
      "accident_density": "Accidents per km² within the buffer, normalised against the 99th-
percentile density across all 500m cells in the dataset.",
      "severity_score": "Severity-weighted mean: (3*fatal + 2*serious + 1*slight) / (3 *
total). Normalised so a cell with all-fatal accidents scores 1.0.",
      "time_risk": "Accident count for the requested hour and day from the accidents-by-time
heatmap, normalised against the maximum cell count.",
      "speed_limit_risk": "Fatal rate percentage for the dominant speed limit band, taken from
severity-by-speed-limit data and normalised against the maximum observed fatal rate.",
      "cluster_proximity": "1.0 if the segment midpoint is within any DBSCAN cluster radius.
Otherwise decays linearly from 1.0 at the cluster edge to 0.0 at 2km beyond the cluster
radius."
    },
    "risk_labels": {
      "0.0–0.2": "Very Low",
      "0.2–0.4": "Low",
      "0.4–0.6": "Moderate",
      "0.6–0.8": "High",
      "0.8–1.0": "Critical"
    }
  }
}
```


Status Codes: `200`